

Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 6_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (≥ 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

Input Format

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

Output Format

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: 4

3 10 20

1 2 50

6 19 100

2 100 200

Output: The optimal profit is 250

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for job
```

```
struct Job {
```

```
    int start, finish, profit;
```

```
};
```

```
// Compare jobs based on finish time
```

```
int compare(const void *a, const void *b) {
```

```
    struct Job *j1 = (struct Job *)a;
```

```
    struct Job *j2 = (struct Job *)b;
```

```
    return j1->finish - j2->finish;
```

```
}
```

```
// Find last non-conflicting job
```

```
int latestNonConflict(struct Job jobs[], int i) {
```

```
    for (int j = i - 1; j >= 0; j--) {
```

```
        if (jobs[j].finish <= jobs[i].start)
```

```

    }
    return -1;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Job jobs[100];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].start, &jobs[i].finish, &jobs[i].profit);
    }

    // Step 1: Sort jobs by finish time
    qsort(jobs, n, sizeof(struct Job), compare);

    // DP array
    int dp[100];

    dp[0] = jobs[0].profit;

    for (int i = 1; i < n; i++) {
        int inclProfit = jobs[i].profit;

        int l = latestNonConflict(jobs, i);
        if (l != -1)
            inclProfit += dp[l];

        dp[i] = (inclProfit > dp[i - 1]) ? inclProfit : dp[i - 1];
    }

    printf("The optimal profit is %d", dp[n - 1]);

    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

0 1 with distance 3

1 2 with distance 1

2 3 with distance 1

0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Input Format

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

Output Format

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output: 0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Answer

```
#include <stdio.h>
```

```
#define INF 99999
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    int dist[10][10];
```

```
    // Initialize matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```
    // Input edges
```

```
    for (int i = 0; i < m; i++) {
```

```
        int u, v, w;
```

```
        scanf("%d %d %d", &u, &v, &w);
```

```
        dist[u][v] = w;
```

```
        dist[v][u] = w; // undirected graph
```

```
    }
```

```
    // Floyd-Warshall Algorithm
```

```

for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print result matrix
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", dist[i][j]);
    }
}

return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her goal is to determine whether there exists a path from one vertex to another for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N , representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the adjacency matrix graph, where $\text{graph}[i][j]$ is either 0 or 1.

Output Format

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

Sample Test Case

Input: 4

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

Output: 0 1 1 1

0 0 1 1

0 0 0 1

0 0 0 0

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[10][10];
```

```
    // Input adjacency matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    // Warshall's Algorithm
```

```
    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
```

```
            }
```

```
        }
```

```
    }
```



```
// Print reachability matrix
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", graph[i][j]);
    }
}

return 0;
} // You are using GCC
```

Status : Correct

Marks : 10/10

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should implement the 0–1 Knapsack Problem algorithm to find the best combination of items to maximize their total value while respecting the weight constraint.

Input Format

The first line contains an integer n , representing the number of items available for selection.

The second line contains an integer limit, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

Output Format

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 2

25

Bag 10 250

Note 15 430

Output: Total value: 680

Total weight: 25

Answer

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    int n, W;
    scanf("%d %d", &n, &W);

    char name[10][50];
    int wt[10], val[10];

    for (int i = 0; i < n; i++) {
        scanf("%s %d %d", name[i], &wt[i], &val[i]);
    }

    // DP table
    int dp[10][1001] = {0};

    // Build DP table
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i-1] <= w) {
                int include = val[i-1] + dp[i-1][w - wt[i-1]];
            }
        }
    }
}
```

```

        int exclude = dp[i-1][w];
        dp[i][w] = (include > exclude) ? include : exclude;
    } else {
        dp[i][w] = dp[i-1][w];
    }
}
}

// Backtrack to find total weight used
int totalWeight = 0;
int w = W;

for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i-1][w]) {
        totalWeight += wt[i-1];
        w -= wt[i-1];
    }
}

printf("Total value: %d ", dp[n][W]);
printf("Total weight: %d", totalWeight);

return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10